PQMigrate: Evidence-Driven and Abstention-Aware Planning for Post-Quantum Cryptographic Migration

Dr. Deepak Uplaonkar, Ratnesh Patnaik, Saravanan Kumar,
Vaibhavi Shah, and Prince Ostwal

Abstract— Post-quantum migration requires identifying cryptographic roles rather than merely algorithm names. PQMigrate is a bounded evidence-linked resolver embedded in an advisory-planning prototype. Evaluation is Python-only: a single developer authored and labeled 72 synthetic RSA cases, including 52 operations, 16 negative controls, and four malformed inputs. The full resolver returns correct roles for 40 operations and no role for 12; these counts reflect the deliberately constructed supported/unsupported mix, not a sampled estimate of generalization. Perfect results on answered cases and authored negative controls likewise do not establish reliability. Method-name AST matching resolves all 12 cases left unanswered by the full resolver, but mislabels 12 key-transport cases and misses eight JWT cases. We report presence flags and usable role answers in separate columns (identical for AST and full inference, since presence requires a known role) and retain historical coupled scores only with explicit qualifications. Three supplementary counterexamples reveal an unrecognized RSA-PSS JWT signature and two random-value encryptions incorrectly inferred as key transport. The pilot evaluates role resolution, not planner abstention, advice correctness, blockers, Go accuracy, or PQC transformation. The contribution is an inspectable implementation and diagnostic pilot with explicit evidence limits; independent real-code evaluation and external comparison remain incomplete.

Index Terms: post-quantum cryptography, cryptographic migration, static analysis, role resolution, cryptographic inventory, reproducibility.

I. INTRODUCTION

Migration to post-quantum cryptography (PQC) is a software-engineering problem as well as an algorithm-selection problem. NIST standardizes ML-KEM for key encapsulation and ML-DSA for digital signatures in FIPS 203 and FIPS 204, respectively [1], [2]. These standards define different cryptographic functions. Finding the token `RSA` is therefore insufficient to select a replacement: a signature interface and an RSA encryption interface do not have interchangeable requirements. Migration and cryptographic-agility research also emphasizes dependencies, deployed interfaces, and coordinated change [3], [4].

SLH-DSA is another standardized signature family under FIPS 205 [5]. FN-DSA/Falcon remains an ongoing NIST standardization effort rather than a finalized equivalent standard [6]. RFC 9964 standardized ML-DSA identifiers for JOSE and COSE in May 2026 [7]; deployment still requires compatible libraries, key representations, peers, and size limits. None of these PQC integrations is evaluated here.

Consider a program that generates an RSA key and later signs an application message. A planner can identify a signature role and suggest evaluating an ML-DSA-based design while retaining requirements for provider support, serialization, verification peers, and tests. If RSA instead encrypts a newly generated session secret, planning concerns key transport and a possible KEM-based redesign. A KEM establishes a shared secret; it is not a general-purpose replacement for an `encrypt(message)` API. An import with no linked use supports an inventory lead but no resolved migration role.

PQMigrate makes this distinction explicit. It produces evidence-linked observations, attempts bounded role inference, applies a versioned knowledge base, and exposes conditional advice or abstention. Missing evidence remains a blocker. Advice does not authorize mutation. The research question is whether this design makes detection coverage, role correctness, and unresolved context measurable and reviewable.

The contributions are threefold. First, we describe an implemented evidence-to-decision architecture with typed findings, versioned rules, deterministic decision traces, and explicit refusal conditions. Second, we report a reproducible synthetic pilot comparing lexical lookup, method-name AST matching, and the full Python RSA resolver, together with two output-masking sensitivity checks. Third, we document engineering validation and specify an independent evaluation protocol for future work. We make no first-of-kind claim, no claim of superiority over external analyzers, and no claim of validated automatic PQC transformation.

II. BACKGROUND AND RELATED WORK

A. Cryptographic discovery and migration planning

NIST's migration project distinguishes cryptographic discovery from interoperability and performance testing [8]. Cryptoscope combines cryptographic knowledge with parsing and data/control-flow analysis to identify cryptographic usage [9]. This is close prior work: operation-aware discovery already exists, so semantic analysis alone is not a sufficient novelty claim. PQMigrate emphasizes the inspectable connection between bounded source evidence, unresolved planning obligations, and abstention. We do not execute Cryptoscope or compare its published numbers with this pilot's numbers.

Hasan et al. organize migration around inventories and cryptographic dependencies, including graph-based case studies [4]. PQMigrate addresses a narrower source-level task: explaining why an observation supports an advisory family or remains unresolved. It does not replace an enterprise dependency inventory or estimate organizational migration cost.

B. API analysis and evidence interchange

CogniCrypt assists developers with cryptographic code generation and static checks of API use [10]. It demonstrates the importance of task context and integration checks; its task and evaluation differ from the Python RSA role-resolution pilot here. It is related work rather than an executed baseline.

CrySL and CogniCrypt_SAST specify and check cryptographic API usage, including allowed call sequences and constraints [11]. CryptoGuard addresses cryptographic misuse detection with context-sensitive analysis [12], and CryptoAPI-Bench supplies benchmark cases for comparing such analyzers [13]. CodeQL provides reproducible crypto queries, including weak-algorithm checks [14]. The IBM-origin Sonar Cryptography Plugin and CBOMkit address cryptographic discovery and inventory presentation [15], [16]. These works preclude treating identity-aware discovery or CBOM export alone as novel. Their analysis goals and supported languages differ; none was executed on this pilot, and no comparative superiority is claimed.

CycloneDX CBOM represents cryptographic assets and their relationships [17], while SARIF provides an interchange format for static-analysis results [18]. PQMigrate uses these formats to carry findings and provenance. Schema validity establishes structural conformance, not the truth or completeness of a finding, downstream ingestion success, or system security.

C. Abstention and coverage

Selective prediction studies the trade-off between answering and rejecting uncertain inputs [19]. PQMigrate adopts the distinction between coverage and correctness among answers. Its rejection policy is rule-based, with categorical evidence labels rather than calibrated probabilities. It does not inherit statistical risk guarantees from selective-classification research. A refusal to guess can still be a missed operation, and reporting both outcomes is necessary.

Chow's reject-option formulation [20] and El-Yaniv and Wiener's selective-classification framework [21] provide foundational distinctions between coverage and error among accepted predictions. Our role non-answer is not a measured planner reject decision, and the authored pilot provides no selective-risk guarantee.

III. PROBLEM DEFINITION AND RESEARCH QUESTIONS

A. Analysis unit and outputs

The system accepts a source tree, a detection registry, and a versioned migration knowledge base. A finding records a primitive, source location, operation, role, protocol context, confidence category, and evidence. A plan adds the selected rule, advisory target family, unresolved requirements, blockers, and decision trace.

The pilot uses a more restricted unit: one Python snippet mapped to one resolver result. It is not a benchmark of every operation within a multi-operation file. The pilot roles are signature, key transport, encryption, and unknown; operations are sign, verify, encrypt, decrypt, and unknown. Protocol labels are JWT and unknown. Because the evaluator defines a predicted operation as a non-unknown role, operation detection and role resolution are coupled in this pilot.

B. Research questions

1. **RQ1:** What presence-positive and usable-role outputs do lexical lookup, method-name AST matching, and bounded identity/context inference produce on the pilot?

2. **RQ2:** What role correctness is obtained at the observed answer coverage, and which cases remain unresolved?

3. **RQ3:** How sensitive are role and protocol metrics to masking session-secret and protocol output information?

4. **RQ4:** What reproducibility and assurance properties are demonstrated by the implemented artifact workflow?

Whether abstention reduces unsafe migration recommendations on independently labeled real code remains open. No completed experiment here directly estimates that rate.

IV. SYSTEM ARCHITECTURE AND REQUIREMENTS

A. Evidence pipeline

The implementation follows six stages: source discovery; parsing and bounded inference; typed CryptoIR construction; knowledge-base matching; advisory planning and tracing; and export or presentation. Python analysis uses the standard AST. Go inventory discovery is supplemented by Tree-sitter-based bounded semantic analysis. Implementation support is distinct from measured coverage: the quantitative study contains Python RSA snippets only.

Table I. Pipeline modules and evidence boundaries. The dashboard presents artifacts rather than adding semantic inference.

Module	Input and output	Evidence boundary
Discovery and parsing	Files to leads and syntax	No target execution during scanning
Role resolver	Imports, assignments, calls to role/context	Bounded scope and patterns
CryptoIR	Findings to typed records and locations	Unknown fields retained
Knowledge and planner	Findings to conditional advice	Versioned rules and blockers
Trace and export	Decisions to JSON, SARIF, CBOM	Provenance and schema checks
Assurance and UI	Evidence to checks and views	Local validation; apply disabled

B. Functional and assurance requirements

The system must distinguish inventory leads from resolved uses, preserve locations and evidence, represent unknown or conflicting roles, identify matching rules and versions, and retain blockers in exported decisions. The planner must not turn an advisory target into permission to modify code. Categorical confidence and priority are not probabilities.

The knowledge loader rejects duplicate identifiers, missing metadata, unsupported taxonomy values, and mutation-enabling rule settings. Current advisory rules require manual intervention. Decision traces record detection, inference, rule matching, blocker assessment, and outcome. Their identifiers hash canonical semantic inputs while

excluding scan identifiers and timestamps. Equivalent pinned semantic inputs therefore preserve trace identity; timestamped artifacts need not be byte-identical across runs.

C. Trust model

Repositories are untrusted inputs. The analyzer parses source rather than importing or executing the target program. This reduces an execution pathway but does not establish parser hardening, resource isolation, or immunity to hostile inputs. Operators must review evidence and migration constraints. Cryptographic security follows neither from a finding nor from a successful test gate.

V. METHODOLOGY, MATHEMATICAL MODEL, AND ALGORITHM ANALYSIS

A. Evidence and role inference

Let an observation be x, its evidence be $E(x)$, and its inferred role be $\rho(x)$. Evidence includes resolved import names, source spans, supported key-generation assignments, supported random-secret assignments, and linked calls in a module or function scope. Let S, T, and C denote signature, key transport, and encryption. A compact model is

$$\rho(x) = \begin{cases} S & \text{if linked signature use} \\ T & \text{if encryption of a tracked secret} \\ C & \text{if encryption, purpose unresolved} \\ \bot & \text{if no supported use or conflicting roles} \end{cases} \quad (1)$$

The encryption case also includes linked decryption of unresolved purpose. Here $\bot$ means unknown. These cases summarize matching rules, not proven runtime behavior. A locally generated random value is the implementation's proxy for key-transport purpose; it does not independently prove later use as a session key.

The Python resolver collects imports and same-scope assignments, records supported RSA key generation and random-secret producers, and examines calls in source-line order. JWT is recognized when `jwt.encode` consumes a tracked RSA key and a literal algorithm string satisfying the implemented `RS` prefix check. This is not a complete JWT algorithm registry or protocol validation. Sign/verify methods on tracked roots support a signature role. Encryption of a tracked random value supports key transport; other linked encryption/decryption receives the broader encryption role.

The prefix excludes RSA-PSS identifiers such as PS256, defined in RFC 7518 [22]. Section VIII reports a PS256 diagnostic non-answer. Random nonce, challenge, or padding bytes can also match the random-value proxy without representing transported key material; two counterexamples are reported separately from the pilot.

Results are aggregated across the snippet. Multiple observed roles cause an unknown result with conflict evidence; otherwise the earliest matching result is selected. This is not a complete per-call inventory. Reassignment, aliasing, arbitrary wrappers, branch feasibility, cross-function flow, and dynamically selected providers are not established. Identity tracking is a syntactic approximation rather than a sound points-to analysis.

B. Planning and abstention

Let K_v be the knowledge base at version v. The planning function is

$$\pi(x, K_v) = \begin{cases} \text{advise}(t,B) & \text{if a compatible rule matches} \\ \text{abstain}(B) & \text{otherwise} \end{cases} \quad (2)$$

where t is a target family and B contains blockers or unresolved requirements. Advice is conditional: a signature role may support ML-DSA assessment, while key transport supports assessment of a KEM/data-encryption construction. Neither establishes provider availability, peer compatibility, key storage behavior, or application semantics.

Automatic application is governed by an assurance predicate

$$G(x) = R(x) \wedge C(x) \wedge I(x) \wedge T(x) \wedge A(x) \quad (3)$$

whose terms denote a known role, supported construction, interoperability evidence, required tests, and operator authorization. A false or unknown prerequisite prevents eligibility. This is a policy contract, not a cryptographic security theorem. Current RSA plans retain `patch_available=false`.

C. Bounded algorithm

```
Input: source snippet and versioned knowledge base
1. Parse syntax; retain parse failure evidence.
2. Collect imports and local assignment observations.
3. Record supported RSA keys and random values.
4. Examine linked calls in source-line order by scope.
5. Resolve a unique role or retain unknown/conflict.
6. Match an advisory rule and record its version.
7. Emit advice or abstention with trace and blockers.
8. Refuse application with unmet prerequisites.
```

Algorithm 1. Evidence-to-plan procedure. The pilot evaluates steps 1-5; the planner adds steps 6-8.

D. Complexity and implementation cost

Let N be the Python syntax-node count and n_s the collected nodes in scope s. Parsing and traversal are ordinarily linear in input size, but the implementation sorts each scope's nodes twice by source line. With bounded identifier lengths and average constant-time dictionary access, one resolver invocation therefore costs $O(N+\Sigma_s n_s \log n_s)$, bounded by $O(N \log N)$ when nodes are counted a bounded number of times across scopes. Memory is $O(N+F)$, where F counts accumulated results. Very long names and malformed-input parser behavior require separate consideration.

Reanalyzing a file for L inventory leads can multiply total work by L; this per-invocation bound is not a proven whole-scanner bound. Go inventory matching also depends on pattern count and source bytes. No scaling experiment or production throughput claim is reported. These bounds motivate profiling and caching studies.

VI. COMPLETED PILOT: EXPERIMENTAL DESIGN

A. Corpus and labels

PQMigrateBench v0.1 contains 72 single-author synthetic Python RSA cases (Table II). Detection and role metrics score 68 parseable cases: 52 labeled operations and 16 negative controls. Four designated malformed inputs are retained separately. Unsupported but syntactically valid operations remain in the 52-operation denominator; they are not removed because the resolver cannot handle them.

Table II. Completed pilot composition. Outcomes describe the resolver, not migration recommendations.

Category	Cases	Full resolver outcome
Direct signature	12	12 correct roles
JWT signature	8	8 correct roles and contexts
Key transport	12	12 correct roles
Asymmetric encryption	8	8 correct roles
Unsupported valid operations	12	12 unanswered, counted as misses
Negative controls	16	16 negative predictions
Malformed inputs	4	Separately accounted
Total	72	68 scored for detection/role

Nominal train, development, and test assignments contain 44, 14, and 14 cases. They are synthetic partitions, not independently collected repository holdouts. The resolver is not a learned model. Naming a partition “test” does not establish independence from author decisions or related scenario leakage. Labels have not undergone independent annotation or adjudication, and inter-rater agreement is unavailable.

Construction directly shapes the headline counts: the 12 unsupported cases were selected by the resolver's author, while the 40 supported cases and 16 negative controls exercise known scenario families. Their proportion determines observed coverage. These results are implementation diagnostics, not estimates of naturally occurring unsupported prevalence or out-of-sample accuracy.

The nominal partitions miss 8/32, 2/10, and 2/10 gold operations; their similar construction replicates the same limitations and supplies no independent validation.

B. Executed baselines and sensitivity checks

The lexical baseline flags an operation when a lowercased snippet contains `rsa`; otherwise it returns unknown. It does not resolve imports and always leaves role and operation labels unknown. It is an intentionally weak inventory heuristic, not a representative implementation of every lexical analyzer.

The AST baseline matches attribute calls named sign, verify, encrypt, or decrypt without resolving receiver identity. It returns a result only when observed method-derived roles are unique. Sign/verify maps to signature; encrypt/decrypt maps to encryption. It does not distinguish transported key material or recognize the JWT wrapper.

The full configuration calls the bounded resolver in Section V. Two post-prediction output masks rewrite key transport as encryption or protocol as unknown. They are metric-sensitivity checks, not component ablations; inference is not rerun with stages disabled. The derivative reporting artifact names them `session_secret_output_mask` and `protocol_context_output_mask`, retaining original historical identifiers solely as provenance. Frozen release evidence is not rewritten.

All configurations use identical case IDs, labels, assignments, and accounting classes. Malformed cases are excluded from detection/role denominators for every configuration, including the non-parsing lexical baseline. The harness does not implement uniform per-case timeout/crash measurement; no comparative robustness result is claimed. Regex-call and external-tool baselines are planned, not executed.

C. Metrics and denominators

On the 68 scored cases, detection uses true positives, false positives, false negatives, and true negatives:

$$P = TP/(TP + FP),\ R = TP/(TP + FN)$$
$$F_1 = 2PR/(P + R) \tag{4}$$

Let O be the 52 gold-operation cases, $A \subseteq O$ those returning a usable, non-unknown role, and $C \subseteq A$ those with the correct role. We report

$$\text{Coverage}_O = |A|/|O|,\ \text{Acc}_A = |C|/|A|$$
$$\text{Acc}_O = |C|/|O| \tag{5}$$

These are gold-operation-conditioned measures. Conditional role accuracy excludes false-positive operations on negative controls and must be read alongside presence errors. Lexical lookup produces presence flags but no usable roles; its conditional role accuracy is undefined, not zero. Table IV separates presence flags, usable role outputs, and correct roles. Historical presence scores for AST and full inference remain coupled to known-role outputs: they are not an independently evaluated presence-detection channel or a fair standalone detection ranking against an inventory-only baseline.

The operation non-answer rate is 1-Coverage_O; it is not the planner's abstention rate. Exact operation/role accuracy uses all 68 scored cases, requiring correct presence and, for gold operations, correct role and operation labels. Protocol accuracy uses the eight cases with known gold context. Error count is the number of cases with at least one scored mismatch, not the number of unsafe recommendations.

VII. LIMITATIONS AND THREATS TO VALIDITY

Construct validity. Snippet-level summaries couple detection to a known-role result. They do not measure complete per-call extraction, planner abstention, or recommendation safety. Random-source tracking is a proxy for key-transport purpose; evidence strength is not calibrated confidence.

Internal validity. One author constructed the corpus and labels, and development had access to scenario families. The lexical baseline is weak; no external analyzer was executed. Output masking is not internal component ablation. Bounded-analysis limitations include reassignment, aliasing,

wrappers, cross-scope flow, and mixed-role aggregation. Tests do not establish soundness.

External validity. Quantitative evidence is synthetic Python RSA only. It cannot establish Go accuracy, real-repository performance, other primitive/protocol coverage, or production dependency behavior. Existing repository scans lack the independent gold labels needed for accuracy claims. The nominal test partition is not a repository holdout.

Statistical conclusion validity. Confidence intervals, significance tests, independent agreement results, and population sampling are absent. Scenario correlation and purposive construction prevent treating snippets as independent natural-code draws. Perfect conditional accuracy on 40 selected answers must not be advertised as general reliability; two of three post-hoc probes already produced wrong role answers (Table V).

Security and deployment validity. Advisory planning does not prove confidentiality, unforgeability, or secure composition. Digest-contract tests provide no PQC interoperability evidence. Application remains disabled, public deployment is unverified, and faculty/security approval and release tagging are incomplete. Concrete integrations must check current standards and implementation errata.

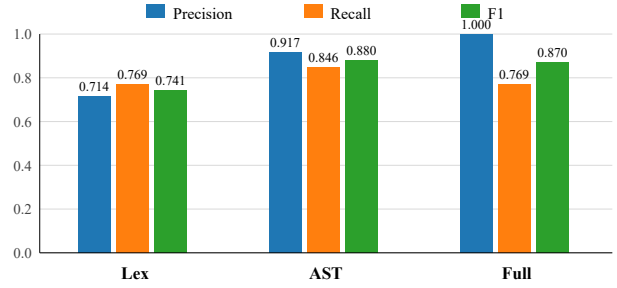
VIII. PILOT RESULTS AND ERROR ANALYSIS

A. Detection and role outcomes

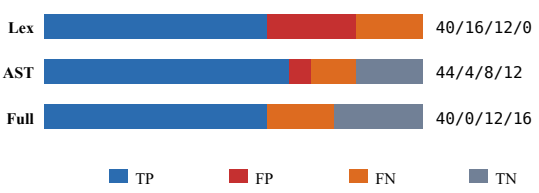
Tables III and IV report completed aggregate results. Values are rounded to three decimals; artifacts retain six decimals and raw predictions.

Table III. Historical presence-positive accounting on 68 cases. AST/full positives require a known role; lexical positives do not. These coupled scores must not be interpreted as an independent detection comparison.

(a) Presence-positive accounting



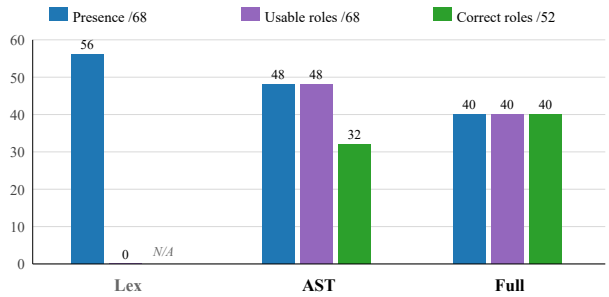
(b) Outcomes; exact tuple is TP/FP/FN/TN



Lex=lexical lookup; AST=method-name matching; Full=identity/context-aware resolver.

Table IV. Separate presence flags and usable role answers. AST's 48 role outputs include four false positives; 44 concern gold operations. Gold-operation role coverage is 44/52 for AST and 40/52 for full inference; end-to-end role accuracy is 32/52 and 40/52 respectively. Lexical lookup has no role-resolution capability.

Graphical view: output counts



Tabular view: exact counts and conditional accuracy

Configuration	Presence positives /68	Usable roles /68	Correct roles /52	Accuracy on answers
Lexical RSA lookup	56	0	N/A	N/A
AST method matching	48	48	32	32/44 = 0.727
Full resolver	40	40	40	40/40 = 1.000

The AST baseline detects four more gold operations than the full resolver but also produces four false positives. Its 0.880 F1 exceeds the full resolver's 0.870. However, it returns 32 correct roles among 44 answered gold operations, versus 40 among 40 for the full resolver. End-to-end role accuracy is consequently 40/52 = 0.769 for the full resolver and 32/52 = 0.615 for AST matching. These are descriptive differences, not statistically established effects.

Importantly, AST matching correctly resolves all 12 operations designated unsupported by the full resolver. Its remaining role errors comprise 12 key-transport cases labeled encryption, and its eight missed gold operations are JWT signatures. Thus stronger identity constraints sacrifice coverage on cases where simpler matching succeeds; the pilot does not establish that every resolver non-answer is beneficial or necessary.

AST succeeds on those 12 cases because it does not check which object receives the call. The same property produces its four false positives on negative controls, so its success there shows that a method name exists, not that the operation is RSA.

All 12 full-resolver errors belong to the unsupported-valid-operation category. Non-answer rate is 12/52 = 23.1% among gold operations. Across 68 scored cases, 28 predictions are operation-negative: 12 missed operations and 16 correctly rejected negative controls. Calling all 28 “abstentions on operations” would conflate outcomes. Exact operation/role accuracy is 56/68 = 82.4%, combining 40 correctly resolved operations and 16 correctly rejected negatives.

B. Sensitivity to output information

Masking session-secret information changes 12 correct key-transport roles to encryption. Detection counts remain unchanged while end-to-end role accuracy falls from 40/52 to 28/52. Detection metrics therefore cannot reveal the role distinction erased by the mask. This does not measure how a separately implemented analyzer without secret tracking would behave.

The protocol mask preserves operation and role predictions but changes eight correct JWT contexts to unknown. Context accuracy falls from 8/8 to 0/8 and error cases rise from 12 to 20. This demonstrates the accounting value of reporting context separately, with no evidence about TLS, SSH, certificates, or other contexts.

C. Post-hoc diagnostic probes

Three additional syntactically valid snippets were analyzed after the pilot was frozen, separately from the frozen 72-case corpus. Their intended purposes were declared by the corpus author, not independently annotated. They test named failure modes rather than estimate prevalence, advice correctness, or security.

Table V. Supplemental counterexamples; no pilot denominator or headline score is changed. Snippets and predictions are retained in `PQMIGRATE_PILOT_DIAGNOSTIC_PROBES.json`.

Diagnostic	Declared operation role	Resolver result
JWT encode with tracked RSA key and PS256	Signature	Unknown
RSA encrypts random challenge bytes	Encryption, not key transport	Key transport
RSA encrypts random padding bytes	Encryption, not key transport	Key transport

These outcomes expose a concrete signature-coverage gap and false-positive purpose inference from random-byte provenance. They invalidate an interpretation that the proxy itself proves transported key material.

D. Failures and unmeasured outcomes

Malformed inputs constitute 4/72 = 5.6% of the corpus. This is a designed-case fraction, not an estimated parser-failure rate on natural code. Diagnostics remain outside the principal score denominator. The current error taxonomy distinguishes presence, role, operation-label, and protocol mismatches; it does not establish every unsupported case's root cause independently.

No bootstrap intervals, independent-annotation agreement, deployment success rates, or false-recommendation rates have been completed. Forty correct roles among 40 answers do not establish zero future error, and zero detection false positives do not establish safe migration advice. A naive case-level interval would not remedy the authored corpus and correlated scenarios.

As a descriptive caution only, if 40 answers were independent identically distributed draws from a specified population, zero observed errors would yield an approximate 95% upper error bound of 3/40 = 7.5% by the rule of three [23] (exact one-sided binomial bound: 7.22%). Those sampling assumptions do not hold here, so neither value is a confidence bound for PQMigrate's real-code error rate.

IX. IMPLEMENTATION ASSURANCE AND REPRODUCIBILITY

A. Completed engineering evidence

The D0-D20 integration gate passed twice from a clean detached checkout of the same implementation commit. Preserved logs record 97 Python tests, 23 legacy scanner checks, five Maven tests, and a successful React/Vite build, together with later milestone scripts. These are software checks, not additional independently labeled research cases.

The preview/verification workflow demonstrates a narrow MD5-to-SHA-256 expression transformation. It binds a preview to a source hash, materializes a disposable copy, checks Python syntax and expected AST structure, and verifies that original source remains unchanged. This does not establish application-level semantic equivalence. The temporary filesystem tree is not a container, virtual machine, or full security sandbox.

The two-party laboratory evaluates a controlled digest-negotiation contract with agreement, fallback, no-common-mode, and tampered-digest scenarios. It does not implement ML-KEM, ML-DSA, or a standardized PQC hybrid protocol. These checks demonstrate the assurance workflow, not successful PQC migration.

Deployment evidence identifies a locally validated WSL CLI, loopback dashboard, local static-site preview, backend tests, and frontend build. There is no public hosting or production-deployment claim. Release tagging and faculty/security review remain pending. Dependency-audit findings require triage before release readiness.

B. Source and artifact identity

The source repository is available at the project URL [24]. The evaluated implementation commit is

`bcfb5e40fbc8eb74e1c919114f54031d03ab44e9`

The SHA-256 of `benchmarks/pqmigratebench_v0_1.jsonl` is

`293fee315c4fbcb95f0d071615ce54a7ac6c9b8b89783a832832c15e3697603b`

The full gate is `bash scripts/validate_d0_d20.sh`; the focused baseline gate is `bash scripts/run_d18_baselines.sh`. The D20 manifest records the source commit and milestone-artifact hashes. Complete logs, artifacts, and checksums were preserved separately under `pqmigrate-release-evidence/bcfb5e4/run1/` and `run2/` in the local environment. This avoids changing the validated source commit to hold regenerated evidence. Local preservation is verified; an immutable public evidence deposit has not been established.

Reproduction requires the documented Python dependencies and Tree-sitter grammar, Java/Maven, and Node/npm dependencies. Historical runs reused the configured project environment. Clean source checkout is supported; a hermetic build from a new machine is not established. Later documentation revisions do not change the identity of the evaluated implementation.

X. DISCUSSION

The pilot illustrates that higher detection F1 can coexist with fewer correct migration-relevant roles. Distinguishing transported key material from generic encryption matters even when both use an encryption call. Preserving unresolved context is useful for review, although it does not itself establish safety.

The diagnostic probes show the limit of that distinction: the random-value proxy labeled two non-key encryptions as key transport, so the 40/40 conditional accuracy applies only to the authored supported families.

Role non-answer has a cost: 12 genuine operations remain unresolved and require investigation. Favorable conditional accuracy is obtained on a deliberately bounded subset. Expanding that subset could improve coverage while introducing wrong answers. Measuring this trade-off on independent code is the next empirical task; it must remain distinct from planner abstention.

The workflow also distinguishes syntax checking, schema validation, source preservation, peer compatibility, and cryptographic correctness. Passing one obligation does not discharge the others. This can organize migration reviews, but reviewer effort and decision quality have not been evaluated in a user study.

XI. PLANNED EVALUATION AND FUTURE WORK

A. Independent corpus and annotation: not yet completed

The next evaluation will target 200-300 reviewed cases, with a working target of 240 across multiple repositories and balanced between Python and Go. Categories will include signatures, key transport, encryption, key establishment/protocols, negative controls, unsupported valid operations, ambiguous evidence, and malformed inputs. Inclusion criteria, revisions, licenses, spans, discovery methods, and sampling decisions will be recorded. Multiple candidate-discovery methods will reduce selection toward existing strengths.

Two human annotators will independently label presence, role, context, support status, and advisory correctness, blinded to tool predictions. Calibration cases will be excluded from final testing. Original labels, disagreements, rationales, and adjudicated labels will be retained. Cohen's kappa [25] will be reported per applicable nominal field with raw agreement and label prevalence. For observed agreement p_o and chance agreement p_e,

$$\kappa = (p_o - p_e)/(1 - p_e) \tag{6}$$

Repository families, forks, and duplicate snippets will remain within a partition. An approximately 60/20/20 allocation will be fixed by repository, with the test set unavailable for rule tuning. Final sizes and allocations will be reported as observed rather than assumed to match targets.

B. Baselines and statistics: not yet completed

A common harness will compare primitive/import lookup, regex-call matching, AST matching, the full pipeline, and a pinned Semgrep Community Edition configuration with published rules [26]. The latter is an external analysis-engine baseline, not a claim that custom Semgrep rules form a complete migration product. All systems will use identical cases, gold labels, output definitions, limits, timeout policies, and failure accounting. Parse failures, unsupported cases, crashes, and timeouts will remain in an explicit ledger.

Analysis will report detection, role accuracy, useful answer coverage, correctness among answers, end-to-end accuracy, planner abstention, false recommendations, parser failures, and runtime by language, category, and held-out split. Unsafe recommendations will require independently adjudicated advisory labels; detection false positives will not substitute for that outcome. Rates will be reported both per recommendation issued and per eligible evaluation case, with explicit denominators.

Planned 95% intervals use 10,000 repository-cluster bootstrap replicates, preserving within-repository dependence and pairing systems on the same resampled repositories. Undefined metrics and small-cluster limitations will be disclosed. Current tables contain none of these future estimates or intervals.

C. Transformations and separate research tracks

Future PQMigrate work will evaluate a bounded RSA signature interface integrated with an ML-DSA provider and a bounded key-establishment fixture using a specified ML-KEM or standardized hybrid construction. Provider versions, test vectors, formats, failure handling, cross-party tests, and performance measurements must precede transformation claims. No such result is part of this manuscript.

The eight-state superposition/public-key investigation is a separate proposed study. It contributes no construction, attack measurement, security proof, or result to this paper. Any future connection requires its own threat model, experiment, and security argument.

XII. CONCLUSION

PQMigrate supplies bounded evidence-linked role resolution within an advisory-planning prototype. Its authored Python-only pilot illustrates reporting distinctions rather than generalization: 40 supported operation cases receive correct roles and 12 selected unsupported cases receive none. Simpler AST matching succeeds on all 12 latter cases, while losing other role distinctions. Separate probes reveal RSA-PSS JWT non-coverage and false key-transport inferences. Planner abstention and recommendation correctness remain unmeasured. Independent labels, real repositories, an external comparison, and validated transformations are required before stronger research or safety claims.

REFERENCES

[1] National Institute of Standards and Technology, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS 203, Aug. 2024, doi: 10.6028/NIST.FIPS.203. https://csrc.nist.gov/pubs/fips/203/final

[2] National Institute of Standards and Technology, *Module-Lattice-Based Digital Signature Standard*, FIPS 204, Aug. 2024, doi: 10.6028/NIST.FIPS.204. https://csrc.nist.gov/pubs/fips/204/final

[3] D. Ott, C. Peikert, and other workshop participants, "Identifying Research Challenges in Post Quantum Cryptography Migration and Cryptographic Agility," Computing Community Consortium workshop report, arXiv:1909.07353, 2019. https://arxiv.org/abs/1909.07353

[4] K. F. Hasan et al., "A Framework for Migrating to Post-Quantum Cryptography: Security Dependency Analysis and Case Studies," *IEEE Access*, 2024, doi: 10.1109/ACCESS.2024.3360412; author manuscript arXiv:2307.06520v2. https://arxiv.org/abs/2307.06520

[5] National Institute of Standards and Technology, *Stateless Hash-Based Digital Signature Standard*, FIPS 205, Aug. 2024, doi: 10.6028/NIST.FIPS.205. https://csrc.nist.gov/pubs/fips/205/final

[6] National Institute of Standards and Technology, "Post-Quantum Cryptography," ongoing standardization status, accessed Oct. 6, 2026. https://csrc.nist.gov/projects/post-quantum-cryptography

[7] M. Prorock and O. Steele, "ML-DSA for JOSE and COSE," RFC 9964, May 2026. https://www.rfc-editor.org/rfc/rfc9964.html

[8] NIST National Cybersecurity Center of Excellence, "Migration to Post-Quantum Cryptography," project resource, accessed Oct. 4, 2026. https://www.nccoe.nist.gov/applied-cryptography/migration-to-pqc

[9] M. Moffie et al., "Cryptoscope: Analyzing cryptographic usages in modern software," arXiv:2503.19531, 2025, doi: 10.48550/arXiv.2503.19531. https://arxiv.org/abs/2503.19531

[10] S. Krüger et al., "CogniCrypt: Supporting Developers in using Cryptography," author manuscript, 2017. https://www.eclipse.org/cognicrypt/resources/publications/knr%2B17cognicrypt.pdf

[11] S. Krüger, J. Späth, K. Ali, E. Bodden, and M. Mezini, "CrySL: An Extensible Approach to Validating the Correct Usage of Cryptographic APIs," in *ECOOP*, LIPIcs, vol. 109, pp. 10:1–10:27, 2018, doi: 10.4230/LIPIcs.ECOOP.2018.10. https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2018.10

[12] S. Rahaman et al., "CryptoGuard: High Precision Detection of Cryptographic Vulnerabilities in Massive-sized Java Projects," arXiv:1806.06881, 2018. https://arxiv.org/abs/1806.06881

[13] S. Afrose, S. Rahaman, and D. D. Yao, "CryptoAPI-Bench: A Comprehensive Benchmark on Java Cryptographic API Misuses," in *IEEE SecDev*, 2019; benchmark artifact. https://github.com/CryptoGuardOSS/cryptoapi-bench

[14] GitHub, "Use of a weak cryptographic algorithm," CodeQL Python query documentation, accessed Oct. 6, 2026. https://codeql.github.com/codeql-query-help/python/py-weak-cryptographic-algorithm/

[15] CBOMkit contributors, "Sonar Cryptography Plugin," source and documentation, accessed Oct. 6, 2026. https://github.com/cbomkit/sonar-cryptography

[16] CBOMkit contributors, "CBOMkit," source and documentation, accessed Oct. 6, 2026. https://github.com/cbomkit/cbomkit

[17] OWASP Foundation, "Cryptography Bill of Materials (CBOM)," CycloneDX documentation, accessed Oct. 4, 2026. https://cyclonedx.org/capabilities/cbom/

[18] OASIS, *Static Analysis Results Interchange Format (SARIF) Version 2.1.0 Errata 01*, OASIS Standard. https://docs.oasis-open.org/sarif/sarif/v2.1.0/errata01/os/sarif-v2.1.0-errata01-os.html

[19] Y. Geifman and R. El-Yaniv, "Selective Classification for Deep Neural Networks," arXiv:1705.08500, 2017, doi: 10.48550/arXiv.1705.08500. https://arxiv.org/abs/1705.08500

[20] C. K. Chow, "On Optimum Recognition Error and Reject Tradeoff," *IEEE Transactions on Information Theory*, vol. 16, no. 1, pp. 41–46, 1970, doi: 10.1109/TIT.1970.1054406. https://doi.org/10.1109/TIT.1970.1054406

[21] R. El-Yaniv and Y. Wiener, "On the Foundations of Noise-free Selective Classification," *Journal of Machine Learning Research*, vol. 11, no. 53, pp. 1605–1641, 2010. https://www.jmlr.org/papers/v11/el-yaniv10a.html

[22] M. Jones, "JSON Web Algorithms (JWA)," RFC 7518, May 2015. https://www.rfc-editor.org/rfc/rfc7518.html

[23] J. A. Hanley and A. Lippman-Hand, "If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators," *JAMA*, vol. 249, no. 13, pp. 1743–1745, 1983, doi: 10.1001/jama.1983.03330370053031. https://pubmed.ncbi.nlm.nih.gov/6827763/

[24] PQMigrate project, "PQMigrate source code and evaluation artifacts," Git repository, evaluated revision bcfb5e40fbc8eb74e1c919114f54031d03ab44e9, 2026. https://github.com/Ratneshp1122/PQMigrate/tree/bcfb5e40fbc8eb74e1c919114f54031d03ab44e9

[25] J. Cohen, "A Coefficient of Agreement for Nominal Scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37-46, 1960, doi: 10.1177/001316446002000104. https://journals.sagepub.com/doi/abs/10.1177/001316446002000104

[26] Semgrep, "Run rules," documentation, accessed Oct. 4, 2026. https://docs.semgrep.dev/running-rules